

phase_08_GAP_AUDIT

Phase 8 (Chutes AI Integration) — Code-Grounded Gap Audit

Revision: 1 **Last modified:** 2026-06-01 **Maintainer:** Engineering Auditor (anti-bluff sweep per CLAUDE-1 / §11.4.68)

Honest completion: ~12% of the Phase 8 roadmap is shipped as real, tested code.

One-line summary: the *security-substrate primitives* that Phase 8 depends on (post-quantum E2EE session/transport, GraVal-style GPU attestation, TEE attestation) are genuinely implemented with real crypto and real-behavior tests in the *security/* submodule — but every Phase-8-specific deliverable (Chutes API client, MinerController, Bittensor wallet/registration, marketplace adapters, Helm charts, node scripts, model router, E2EE proxy wiring, GraVal CUDA kernel) is MISSING. No pkg/chutes, pkg/marketplace, pkg/bittensor exists; no code references chutes, bittensor, TA0, metagraph, or an OpenAI-compatible inference endpoint.

3-Axis Package Status (Refreshed 2026-06-04, HXC-939)

Why this section exists: “exists” ≠ “used”. wired is **measured** via `go list -deps ./cmd/... | grep -Fx <module-path>/<pkg>` (module = `github.com/HelixDevelopment/helix_cluster`), not assumed. **“Completed (registry) ≠ wired”** — Completed only means source+tests exist; it does NOT prove a shipped binary reaches it. Note: *security/pkg/** lives in the **separate `vasic-digital/security` module**, so it is **not** in HelixCluster’s `cmd/` dep graph by construction — those primitives are reusable building blocks, not wired Phase-8 features.

Package	implemented	wired (reachable from cmd/)	tested
pkg/chutes	NO	n/a	n/a
pkg/marketplace	NO	n/a	n/a
pkg/bittensor	NO	n/a	n/a
	yes		yes

Package	implemented	wired (reachable from cmd/)	tested
security/pkg/e2ee (separate module)		NO (not imported by any helix cmd)	
security/pkg/gpuattest (separate module)	yes	NO (not imported by any helix cmd)	yes
security/pkg/attestation (separate module)	yes	NO (not imported by any helix cmd)	yes
pkg/gpuattest (local placeholder shim)	yes	NO (orphaned)	yes
pkg/fiber (8C miner [?] validator transport)	yes	NO (orphaned)	yes
internal/llm (model router — stub Inference)	yes	yes	yes
internal/gpu (local alloc; no Phase-8 hooks)	yes	yes	yes
pkg/scheduler (Omega core; no carbon/compliance)	yes	yes	yes

Orphan callouts: the local pkg/gpuattest shim and pkg/fiber are implemented+tested but reachable from **no** binary. The genuine crypto primitives in the security/ submodule are tested but, being a separate module, are not part of any HelixCluster binary's dependency graph — so no Phase-8 consumer integrates them yet. internal/llm is wired but its Inference is a self-labelled stub, so package-wired ≠ feature-working here.

Method & caveats

- Searched pkg/ internal/ cmd/ api/ security/ for: chutes, bittensor, subtensor, metagraph, TA0, coldkey/hotkey, yuma, graval, marketplace, mlkem, e2ee, attestation, llm.chutes.ai, text/event-stream. Only attestation/e2ee (in security/) and incidental matches hit.
- security/ is the vasic-digital/security submodule. Its packages compile and tests PASS from the submodule root (cd security && go test ./pkg/e2ee/ ./pkg/gpuattest/ ./pkg/attestation/ → all ok). They are NOT yet imported by any HelixCluster Phase-8 consumer (no pkg/chutes to wire them into), so they are reusable building blocks, not an integrated feature.

- CLAUDE-1 / §11.4.68 gate: NONE of the Phase 8 success criteria (TAO on-chain, tokens through validator scoring API, GraVal pass-rate, E2EE proxy audit log, P99 first-token latency) can be evidenced — the producing code does not exist.

Deliverable status table

Deliverable (roadmap §)	Status	Evidence file:line	Notes
pkg/chutes MinerController + GraValVerifier (P0 #1)	MISSING	(no pkg/chutes)	No directory, no symbol. Critical-path revenue gate unstarted.
Bittensor wallet + subnet registration (P0 #2)	MISSING	(grep bittensor/ coldkey/TA0 → 0 hits)	No wallet, no subtensor client, no registration.
Custom HelixGepetto dual-resource strategy (P0 #3)	MISSING	internal/gpu/ manager.go	GPU manager does plain allocation/best-fit/offline; no cost weighting, no Helix-PoW-vs-Chutes split, no load-based hard-cap.
helm/ helixcluster- chutes base chart (P0 #4)	MISSING	(no helm/ dir)	No Helm charts in repo.
Node prep/ deploy/health/ verify scripts (P0 #5)	MISSING	scripts/ (no chutes-*.sh)	None of chutes-node-prep.sh, chutes-miner-deploy.sh, chutes-health-monitor.sh, chutes-verify.sh.
Chutes API Go client + streaming (P1 #6)	MISSING	(grep llm.chutes.ai/ text/event-stream → 0)	No OpenAI-compatible client, no SSE streaming, no fallback chain.
pkg/e2ee ML- KEM-768 proxy (P1 #7) — primitive	PARTIAL	security/pkg/e2ee/ package.go:18–330, transport.go:31–77	Real hybrid PQC: ML-KEM-768 (FIPS 203) + HKDF-SHA256 + AES-256-GCM, nonce-reuse/replay rejection, framed transport. 14 real tests PASS. BUT: it is a session/transport library, not the <i>proxy</i> the roadmap specifies

Deliverable (roadmap §)	Status	Evidence file:line	Notes
			(no Chutes-request interception, no gzip pipeline, no audit log, not wired to internal/gateway). Roadmap names ChaCha20-Poly1305; impl uses AES-256-GCM (a deliberate, documented choice — equivalent AEAD, but a spec deviation to note). Real challenge/response attestation: HMAC-SHA256(key, nonce descriptor), iterated difficulty, single-use nonce, expiry, replay rejection, constant-time compare. 13 real tests PASS. HONEST SCOPE (lines 13-21): software only — proves possession of a device <i>secret</i>, NOT physical GPU silicon. The genuine GraVal CUDA “proof of consecutive VRAM work” kernel is absent (DEFERRED — needs GPU). Not consumed by any miner controller. Real software TEE-document model: ed25519-signed measurements + nonce + policy + freshness, QuoteSigner seam for real hardware. Tests PASS. Hardware-rooted quote (TDX/NVIDIA CC) DEFERRED — needs hardware. No sek8s deployment.
GraVal GPU attestation (P0/P1) — primitive	PARTIAL	security/pkg/gpuattest/package.go:22-323	
TEE attestation (TDX/NVIDIA-CC, sek8s) (P2 #11) — primitive	PARTIAL	security/pkg/attestation/attestation.go:1-40+	
values-models.yaml 8	MISSING	(no helm/)	No model deployment manifests.

Deliverable (roadmap §)	Status	Evidence file:line	Notes
model configs (P1 #8)			
pkg/ marketplace UnifiedManager + Chutes adapter (P1 #9)	MISSING	(no pkg/marketplace)	No MarketplaceAdapter interface, no composite scoring, no revenue optimizer.
io.net / Akash / Salad adapters (P2 #10)	MISSING	(no pkg/marketplace)	Unstarted.
internal/llm model router (latency/ throughput/ quality/cost)	MISSING	internal/llm/ manager.go:133–158	Manager is a registry with an explicitly- labelled stub Inference (synthetic response). Real contract guards (must be registered+loaded, non-empty prompt) exist and are tested, but no router, no Chutes backend, no SSE.
internal/gpu GraVal hooks / MIG / dual- workload split	MISSING	internal/gpu/ manager.go, internal/ gpu/ manager_extra_test.go	Solid allocation logic with real tests, but zero Phase-8 extension (no attestation hook, no MIG profile mgmt, no capacity reservation).
Hybrid PQC TLS node-to- node (P2 #12)	MISSING	—	security/pkg/e2ee provides the KEM primitive but no node- to-node TLS integration (X25519+ML- KEM-768) exists.
Carbon-aware scheduler + EU AI Act compliance pipeline (P2 #13)	MISSING	pkg/scheduler/	Omega scheduler present from earlier phases; no carbon signal, no compliance doc generator, no export-control tier verification.
HelixQA Challenges (miner/E2EE/ marketplace/ GraVal)	MISSING	challenges/	No Phase-8 challenge scenarios; none could pass without the producing code.

TOP IMPLEMENTABLE GAPS (Go, no new infra/hardware)

Ordered by value-per-effort. Each is buildable in pure Go against the *already-shipped* security/ primitives, with mutation-pairable acceptance tests, and needs no GPU/Bittensor/cloud to be a real, tested feature.

- 1. Chutes OpenAI-compatible API client (non-streaming first).** Target: pkg/chutes/client.go. Spec: a Client wrapping net/http that targets a configurable base URL (CHUTES_BASE_URL, default https://llm.chutes.ai/v1) with a cpk_bearer token from env, exposing ChatCompletion(ctx, req) (Resp, error) over the OpenAI /chat/completions schema, with context deadline, typed error decoding (429/5xx → typed retrievable error), and a fallback-model chain ([]string tried in order on retrievable failure). No real key needed for the test — run against an httptest.Server that speaks the OpenAI JSON shape. Acceptance test (mutation-pairable): assert the client sends Authorization: Bearer <token>, honors the fallback chain (first model → 503, second → 200), and surfaces the assistant message. Mutation: drop the Authorization header in source → test must FAIL; drop fallback-advance logic → multi-model test must FAIL.
- 2. SSE streaming decoder for Chutes chat.** Target: pkg/chutes/stream.go. Spec: StreamChatCompletion(ctx, req) (<-chan Delta, <-chan error) that parses text/event-stream (data: {json}\n\n, terminating on data: [DONE]), emitting incremental Delta tokens; cancels cleanly on ctx done; bounds line length. Test against an httptest.Server streaming a fixed SSE script. Acceptance test: feed a 5-chunk SSE script, assert reassembled text + that [DONE] closes the channel + that context-cancel stops mid-stream. Mutation: ignore [DONE] sentinel → channel-close test FAILs; swallow ctx-cancel → cancel test FAILs (or deadlocks under -timeout).
- 3. E2EE inference envelope wiring pkg/chutes [?] security/pkg/e2ee.** Target: pkg/chutes/e2ee_proxy.go. Spec: a thin E2EEProxy that, for requests flagged TEERequired, performs the e2ee handshake (e2ee.NewInitiator/Respond/Complete), seals the request body and opens the response via e2ee.Session, and writes a structured audit-log line (request id, sealed bool, byte counts) — satisfying the roadmap’s “E2EE proxy audit log” sink-side evidence. Pure in-process; loopback io.ReadWriter via e2ee.Transport. Acceptance test: round-trip a request body through initiator+responder sessions over an in-memory pipe, assert plaintext recovered, ciphertext on the wire ≠ plaintext, and an audit record emitted with sealed=true. Mutation: make the proxy pass plaintext through unsealed → “ciphertext ≠ plaintext” assertion FAILs; suppress the audit write → audit-record assertion FAILs.
- 4. GraVal-style miner attestation handshake controller.** Target: pkg/chutes/attest.go (consumes security/pkg/gpuattest). Spec: a Verifier-side RegisterAndChallenge(deviceID, key) + Admit(resp) flow and a Prover-side Attest(challenge) that wraps gpuattest.SoftwareProver/Verifier, plus a BatchVerify([]Response) (passRate float64) returning the roadmap’s GraVal pass-rate KPI. Software-only; the real CUDA kernel stays DEFERRED but this is the integration seam + metric. Acceptance test: register N devices, run N attestations, assert BatchVerify pass-rate ==

1.0; tamper one response, assert pass-rate drops + that device rejected with `ErrResponseMismatch`. Mutation: skip the constant-time compare / always-admit → tampered-device test FAILs.

5. **MarketplaceAdapter interface + Chutes adapter + composite scorer.** Target: `pkg/marketplace/`. Spec: type `Adapter` interface { `Name()` string; `Quote(ctx, GPUSpec)` (`Offer`, error) }; an in-process `ChutesAdapter` returning a deterministic `Offer{PriceUSD, AvailPct, LatencyMs, ThroughputTps}`; a `RevenueOptimizer.Best(offers)` implementing the roadmap's composite score (price 30% / availability 30% / latency 20% / throughput 20%, with a $1.5\times$ TEE multiplier flag). No external marketplace needed — adapter returns configured/fixture data; the *scoring* is the real, testable logic. Acceptance test: feed 3 offers with known fields, assert the optimizer picks the correct weighted winner and that the TEE multiplier flips the winner when set. Mutation: invert a weight sign / drop the TEE multiplier → winner-selection test FAILs.
6. **Dual-workload capacity reservation in internal/gpu.** Target: `internal/gpu/reservation.go`. Spec: extend `Manager` with `ReserveForHelixPoW(fraction float64)` and a `ChutesCapacity()` Stats that reports only the non-reserved remainder, plus a hard-cap rule: when reported Helix load > 80%, `ChutesCapacity` returns zero available (the roadmap's Gepetto starvation guard) — all in-memory, no scheduler rewrite. Acceptance test: reserve 0.8, set load 0.85, assert `ChutesCapacity().Available == 0`; set load 0.5, assert non-zero. Mutation: remove the >80% hard-cap branch → starvation-guard test FAILs.

DEFERRED (external infra / hardware / Zig / GPU kernels)

- **GraVal CUDA “proof of consecutive VRAM work” kernel** — requires real NVIDIA GPU + CUDA toolchain + `libgraval-miner.so`. Software stand-in exists (`security/pkg/gpuattest`); silicon-rooted proof cannot be done in CI.
- **Bittensor mainnet/testnet wallet, subnet registration, TAO emission, metagraph/Yuma queries** — requires a live subtensor node and ~600 TAO for subnet ops; on-chain evidence (`btcli/explorer`) is unobtainable without the network.
- **Real Chutes SN64 end-to-end (tokens through validator scoring API, P99 first-token latency, 100+ nodes on metagraph)** — requires a funded `cpk_key` + live Chutes network; client (#1/#2) can be built/tested against `httptest`, but the KPI evidence is external.
- **Intel TDX + NVIDIA CC hardware attestation (sek8s), LUKS root, Cosign admission, K3s GPU bare-metal** — requires TEE-capable hardware + a K3s cluster. Software attestation model exists (`security/pkg/attestation`); hardware quote chain is deferred.
- **Helm chart real-cluster deploy + values-models.yaml model serving (vLLM/SGLang/AWQ)** — charts are authorable in-repo (a smaller gap), but proving they serve models needs GPU nodes; mark the manifest authoring implementable, the serving-proof deferred.